Shared vs Dedicated Databases

shared schema, schema per tenant, or a database each

Watch the 4-min video

In one line: multi-tenant data lives in a shared schema keyed by `tenant_id`, a schema per tenant, or a database per tenant; mature products mix all three behind a **tenant catalog**, and design for the day a big tenant must move.

1 The tenant that outgrew you

Your largest customer now owns **40%** of the rows in the shared database. Its nightly reports saturate I/O and slow every other tenant, and its new contract requires a dedicated database in Germany. You must move it with no downtime and no lost rows. If `tenant_id` is on every row and the application already looks up where each tenant lives, this is a routine operation. If tenant data is mixed with global data, or tenant IDs are implied by joins, it is a months-long project.

2 Three data models

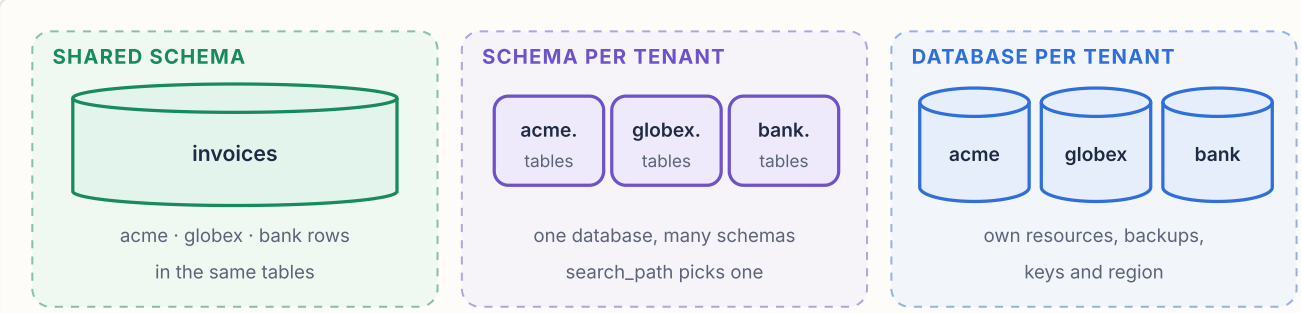


Figure 1. Same data, three layouts. Isolation, cost and operational load all rise from left to right.

Shared schema. One set of tables; every table has `tenant_id`, and every primary key and index starts with it, e.g. `PRIMARY KEY (tenant_id, id)`, so a tenant's rows are clustered and queries never scan other tenants. One migration updates everyone. This is what Citus does at scale: tables distributed by `tenant_id`, so one tenant's rows sit on one node and joins stay local. The risk is a missing `WHERE tenant_id = …`, so use Postgres **row-level security** as a second wall:

```
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoices FORCE ROW LEVEL SECURITY;   -- applies to the owner too
CREATE POLICY tenant_isolation ON invoices
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
-- per transaction, from the verified token (lesson 43):
SET LOCAL app.tenant_id = '6f1c...';
```

Figure 2. RLS policy. FORCE matters: table owners bypass RLS by default, as do superusers and roles with BYPASSRLS. Use a non-owner app role.

Schema per tenant. One database, a schema per tenant (`acme.invoices`, `globex.invoices`), selected with `search_path`. Per-tenant backup and restore (`pg_dump -n acme`) and per-tenant custom columns become easy. The cost is the catalog: 5,000 tenants × 60 tables is 300,000 tables plus their indexes, which slows migrations, `pg_dump` and backups, and every connection caches catalog entries it touches. Watch connection pooling too: with PgBouncer in transaction mode, a plain `SET search_path` can leak to

the next client of that server connection; use `SET LOCAL` inside each transaction or schema-qualified names.

Database per tenant. Strongest isolation: separate CPU and I/O, backups, encryption keys, maintenance windows and even regions; noisy neighbours disappear and restoring one tenant never touches another. But everything scales with the tenant count. Connections are the classic trap: 20 app pods × 1,000 databases × a pool of 5 is 100,000 connections. Use lazily opened, small pools, a proxy (PgBouncer, RDS Proxy), or pooled capacity such as Azure SQL **elastic pools**, where many databases share one vCore budget. Migrations become a fleet operation: record the schema version per database, run migrations in waves, and keep code compatible with both the old and new schema while the fleet catches up (expand, then contract).

	Shared schema	Schema per tenant	Database per tenant
Isolation	logical (code + RLS)	namespace	physical
Cost per tenant	lowest	low	highest
Schema migration	one run	N schemas	N databases, in waves
Restore one tenant	hard (filter a full backup)	easy-ish	easy
Noisy neighbour	yes	yes (same server)	no
Scale limit	table size, hot tenants	catalog size	connections, ops effort
Tenant count it suits	thousands to millions	hundreds to low thousands	tens to hundreds (or automated fleets)

3 How to choose: count, size, compliance

Tenant sizes almost always follow a power law: a few tenants hold most of the data. So the honest answer is a mix. Many small tenants go into a shared schema; a few large or regulated tenants get dedicated databases; schema per tenant is a middle step when customers need per-tenant restore or customisation. A **tenant catalog** records where each tenant lives, and the data access layer picks the connection for every request.

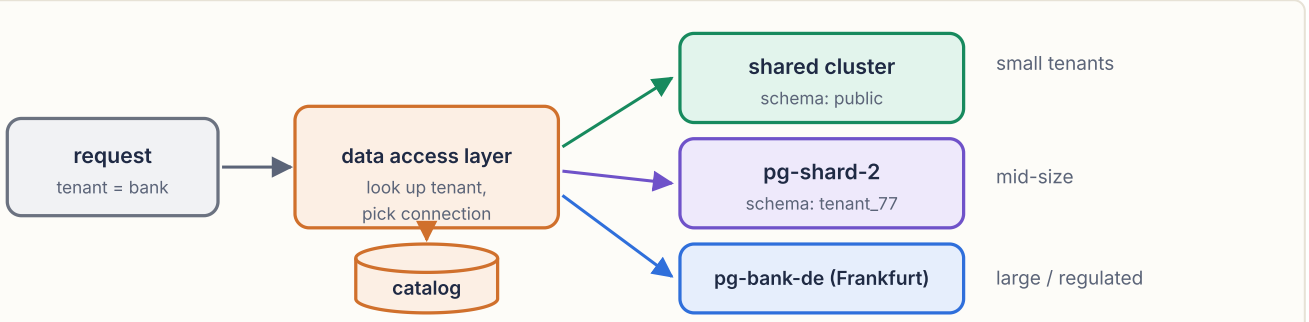


Figure 3. The catalog (tenant → cluster, database, schema, region, status) is tiny, read on every request, and therefore cached.

Keep three rules everywhere, even in dedicated databases: **tenant_id on every row** (so moving is a filter, not a rewrite), **globally unique IDs** such as UUIDs or ULIDs (two databases' sequences would collide when you merge or move), and **no cross-tenant foreign keys or joins**, so one tenant's data can be lifted out cleanly.

4 Design for moving tenants

Copying one tenant is a filtered export, but the tenant keeps writing while the copy runs. The standard answer is **snapshot + change data capture**: record the log position (Postgres LSN, MySQL binlog position), bulk-copy a consistent snapshot, then replay every change after that position, filtered by tenant. Debezium on the WAL, or Postgres logical replication, which since version 15 supports **row filters** on a publication (`CREATE PUBLICATION bank_pub FOR TABLE invoices WHERE (tenant_id = '…')`), both work. Make the apply step idempotent (upsert by primary key), so replaying a change twice is harmless.

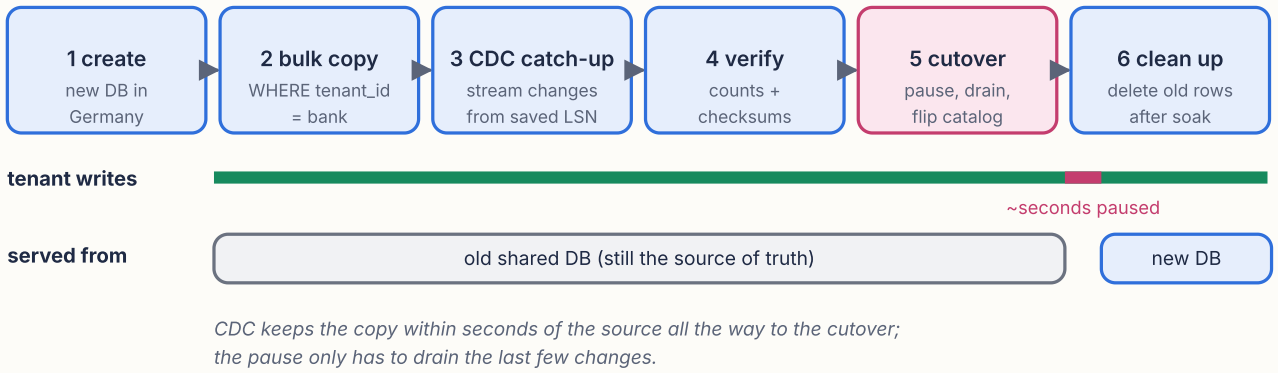


Figure 4. Zero-downtime tenant move. Only step 5 pauses the tenant, for seconds, and only that tenant.

WORKED EXAMPLE: MOVING THE BIG TENANT TO GERMANY

(1) Create `pg-bank-de` in Frankfurt with the same schema version. (2) Note the current LSN, then copy every table `WHERE tenant_id = 'bank'` from a consistent snapshot. (3) Stream changes after that LSN, filtered to the bank, until replication lag is under a second. (4) Verify: row counts per table and checksums over ordered rows (e.g. a hash of each table's primary keys and updated_at). (5) Set the bank's catalog status to `migrating`, which makes the API hold or reject its writes with a retry; wait until lag is zero; flip the catalog to `pg-bank-de`; invalidate cached catalog entries; resume. The pause is seconds and only the bank sees it. (6) Keep the old rows for a soak period as a rollback path, then delete them in small batches. Don't forget the bank's files in object storage, its search index documents and its cache keys.

COMMON MISTAKES

- Tables without `tenant_id` ("it's implied by the parent row"): you can't filter them for a move.
- RLS enabled but the app connects as the table owner, so the policy never applies.
- Database per tenant with a full connection pool per database in every pod.
- Flipping the catalog before change data capture has fully drained, losing the last writes.

5 Key terms

Shared schema: all tenants in the same tables, keyed by tenant_id

Schema per tenant: one database, a namespace each

Database per tenant: a physical database per tenant

Tenant catalog: tenant → where its data lives

Row-level security: database-enforced row filter per session

Change data capture: streaming committed changes from the log

6 Interview questions

Q1. How do you choose a multi-tenant data model? BASIC
Shared schema with tenant_id for many small tenants, schema per tenant for moderate isolation, a database per tenant for large or regulated tenants; most systems mix them with a tenant catalog for routing.

Q2. Why should every index in a shared schema start with tenant_id? BASIC
Every query is per tenant, so leading with tenant_id keeps lookups and scans inside one tenant's rows and makes the data easy to shard by tenant.

Q3. How do you move a big tenant to its own database? SCENARIO
Bulk copy by tenant_id from a snapshot, sync later changes with change data capture, verify counts and checksums, then briefly pause that tenant's writes, flip the catalog and resume.

Q4. You have 2,000 tenants, each in its own database. What breaks first? SCENARIO
Usually connections and operations: pools per database per pod explode, and migrations, monitoring and backups must run as fleet jobs. Use pooled proxies or elastic pools and wave-based migrations.

Q5. RLS is enabled, but a query still returns other tenants' rows. Why? TRICKY
The session's role bypasses RLS: it is the table owner without FORCE ROW LEVEL SECURITY, a superuser, or has BYPASSRLS; or the tenant setting leaked from a pooled connection.

Q6. Why do you need globally unique IDs in a sharded multi-tenant system? TRICKY
Moving or merging tenants copies rows between databases; per-database sequences would collide, while UUIDs or ULIDs move as they are.

Q7. What's the downside of schema per tenant at large scale? TRICKY
Catalog size: hundreds of thousands of tables slow migrations, dumps and connection start-up, and every schema change runs thousands of times.

Next: SD-R-003 · Tenant-Aware Authorization backendroadmap.com